GIT AND GITHUB: BRANCHING, COMMITS, MERGING AND REMOTE WORKFLOWS

Asrita Maurya Battula — 261090051430

Formula Manipal Driverless Taskphase
Manipal Institute of Technology, Manipal, Karnataka, India

Abstract— This report summarizes practical Git and GitHub work covering local version control, branching, commits, merging, rebasing, conflict resolution, logs, stashing, force pushes, and remote operations. The Main and Remote sections of Learn Git Branching were completed and the Task One solution was integrated into a repository.

Keywords— Git, GitHub, version control, branching, commit, merge, rebase, remote, pull, push, conflict resolution

I. INTRODUCTION

Git is a distributed version-control system for tracking changes and coordinating work. GitHub hosts Git repositories remotely for collaboration, backup, code review, and sharing. Git stores project history through commits, while branches are movable references to that history.

II. OBJECTIVES

The objectives were to practice branching and commit history, understand logs and reflogs, merge and rebase branches, resolve conflicts, use fetch/pull/push with a remote, and study stashing and force pushing.

III. GIT CONCEPTS

Repository: Contains Git objects, history, and references. **Working tree:** Current checked-out files. **Commit:** A snapshot of staged changes with metadata and parent references. **Branch:** A movable reference to a commit. **HEAD:** Identifies the current position.

IV. HOW COMMITS ARE STORED AND TRACKED

Git stores commits as objects in the repository's object database. A commit points to a project snapshot (tree), records metadata, and points to its parent commit(s). Branch names and HEAD act as references to commits. This forms a history graph rather than a simple list. The command **git log** displays commit history, while **git show <commit>** displays a commit and its changes.

V. LOCAL REPOSITORY WORKFLOW

A basic workflow is: **git init** → **git status** → **git branch feature** → **git checkout feature** → **git add .** → **git commit -m "Add feature"** → **git checkout main** → **git merge feature**. Editing happens in the working tree, *git add* stages the next snapshot, and *git commit* records it.

VI. LOGS AND REFERENCE LOGS

git log shows the commit history reachable from the current reference. **git reflog** records local movements of references such as HEAD and branch tips, so it can help locate commits after operations such as reset or rebase. Reflogs are local and are different from the normal commit history.

VII. MERGING, REBASING AND MERGE CONFLICTS

Merging combines histories and may create a merge commit. **Rebasing** reapplies commits onto a new base and creates new commit identities, so shared history should be rewritten deliberately. When a merge conflict occurs, inspect the marked file, choose the required content, remove the conflict markers, stage the resolved file with **git add**, and complete the merge.

VIII. REMOTE REPOSITORY WORKFLOW

A local repository can be connected to GitHub through a remote such as **origin**. **git fetch** downloads remote updates without integrating them; **git pull** fetches and integrates them; and **git push** transfers local commits and references to the remote. Common commands include **git remote add origin <repository-url>**, **git fetch origin**, **git pull origin main**, and **git push -u origin main**.

IX. STASHING AND FORCE PUSHING

git stash temporarily shelves uncommitted changes so the working tree can be cleaned without making a commit. **git stash list**, **git stash pop**, and **git stash apply** retrieve stashed work; *pop* removes the applied stash, while *apply* keeps it. A force push, **git push --force** (or safer **--force-with-lease**), replaces remote branch history with local history and can discard commits already pulled by others.

X. LEARN GIT BRANCHING PRACTICE

The Main section was completed, covering commits, branching, merging, rebasing, detached HEAD, relative references, reset, revert, cherry-pick, and interactive rebase. The Remote section was also completed, covering clone, fetch, pull, push, tracking branches, push arguments, and diverging histories. The exercises reinforced how local branches relate to remote-tracking

branches.

IMPORTANT GIT COMMANDS

Command	Purpose
git branch <name>	Creates a branch.
git checkout <name>	Switches branches.
git add .	Stages changes.
git commit -m "msg"	Records staged changes as a commit.
git log / git show	Views history / commit details.
git reflog	Shows local reference movements.
git merge <name>	Combines another branch.
git rebase <target>	Replays commits on a new base.
git fetch / git pull	Downloads / downloads and integrates remote changes.
git push	Transfers local commits/references to remote.
git stash	Temporarily stores uncommitted changes.

XI. OBSERVATIONS

Git is best understood as a graph of commits and movable references. Merge preserves divergent history, while rebase rewrites a sequence of commits onto another base. Logs show commit history and reflogs help recover references after operations such as reset or rebase. Stashing separates unfinished work from committed history, while force pushing changes remote history and therefore requires care.

XII. CONCLUSION

The practical work developed an integrated understanding of Git and GitHub. Branching, commits, logs, reflogs, merging, rebasing, conflict resolution, stashing, fetching, pulling, pushing, and force pushes were studied as parts of a single version-control workflow.

REFERENCES

[1] Git Documentation, “Git Reference.”
[2] GitHub Docs, “GitHub Documentation.”
[3] Learn Git Branching, “Learn Git Branching” interactive tutorial.